自作言語で作った自作 CPU の上で自作 Ruby が動かない

全部自分のコードのとき、どこを疑うか

284km ・ YAPC::Tokyo 2026

最初に、ひとつ選んでください

このあと出てくる 3 つの容疑者のうち、犯人はどれか

最後に答え合わせをします

3 つの層、全部自分のコード

1. 自作 Ruby 処理系（4 万行超）
2. 自作コンパイラが出した RISC-V の機械語
3. 自作 CPU エミュレータ

Ruby 処理系も CPU エミュレータも、自作言語 Mere で書いた

圧縮も PNG もフォントも、このスライドの道具も自作

CPU を 32 ビットから 64 ビットに広げた

puts 1 + 1

が通らない

定石どおり、境界はテストしてあった

層	何と比べたか	結果
1. Ruby 処理系	本家 ruby 3.2.2、162 本	バイト単位で一致
2. 機械語	同じプログラムを C に変換して実行	64 / 32 ビットとも不一致 0
3. CPU エミュレータ	同じ機械語を QEMU で実行	7 / 7 一致

2026 年 9 月初め。全部、緑

境界のテストが言えること

試したプログラムでは、一致する

「その層が正しい」とは、言っていない。容疑者は 3 人とも残る

他人の層なら、材料が外から来る

・issue、既知のバグ、changelog

・「みんな使っているから、たぶん正しい」

全部自分のコードだと、それが 1 つも来ない

見当 1: 直前に手を入れた場所

64 ビット化で、関数ごとの「変数の置き場」の大きさを書き換えた

→ 置き場の数を数え間違えて、隣を踏んでいる

見当 2: 前に一度壊れた仕組み

関数を呼ぶ前に値を退避し、戻ったら書き戻す。ここは一度壊れていた

→ 呼ばれた側が、書き戻し損ねている

どちらも、証拠つき

そして 2 回とも外れた

見当 1 を、検査に変える

確保した置き場の数と、使った数を、コンパイラ自身に比べさせた

reserved 2, used 17

当たっていた。直した。それでも puts 1 + 1 は通らない

見当 2 も、検査に変える

CPU エミュレータに、呼ぶ前の値を覚えさせ、戻ったときに照合させた

一度も鳴らない

外れが確定した。分岐が 1 本ずつ閉じる

観察が、食い違い始めた

・値を出力するコードを足すほど、結果が互いに矛盾した

・推理の結論:「壊れた値は、メモリのどこかにある」

そこで推理をやめて、出来上がった機械語を読んだ（逆アセンブル）

mv a0, s10

メモリからではなく、CPU 自身の置き場（レジスタ s10）から取っている

その関数で s10 に書く命令は 4 つだけ。1 つが、例外から戻るときの書き戻しだった

例外から戻るときの、値の退避先

意図	実際	
確保した区画	15 個ぶん（0〜14）	15 個ぶん（0〜14）
値の退避先	5〜14	20〜29

区画の外は、まだ誰にも渡していない空き地。後から払い出されて、上書きされる

壊れていたのは 1 行

前: 20 + i × 4　先頭から 20 バイト目

後: (20 + i) × wsz　先頭から 20 個目

32 / 64 ビット両対応のための、機械的な置き換え

答え合わせ

2. 自作コンパイラが出した機械語

最初の見当「直前に手を入れた場所」は、方向としては合っていた

では、なぜ緑だったのか

テストは、ずっとあった。でも、失敗する処理の中でメモリを確保していなかった

間違った場所から書き戻すのと

まだ誰も書いていない場所から書き戻すのは

出力が同じ

持ち帰ってほしいこと

自分が正しいと思っていることを、自分以外が判定できる形に置き換える

1. 疑いは、検査にする
2. 黙っている検査を、信じすぎない
3. 観察が食い違ったら、成果物を読む

ここからは、それぞれの事件簿

1. 疑いは、検査にする

事件簿: 文字の d に、横棒が出る

・自作のスライド道具で、30 ピクセルの d の上に、無い線が描かれた

・原因は自作のフォント処理。曲線の向きが変わる点で、線を 1 回多く数えていた

・本物のブラウザとの比較テストは 297 / 297 で通っていた。30 ピクセルの例が無かった

正解を持たない検査

・文字を左右反転して描いても、塗る場所は鏡写しで同じはず

・答えを誰かに用意してもらわなくても、自分自身と比べられる

・古いコードで 87 点、直したコードで 0 点

・最初に見つけたのは、知らなかった組み合わせ（g の 12 ピクセル）

事件簿: 3D の描画が、2 フレーム目で落ちる

・メモリの規則をコンパイラで変えた。3 版続けて出した

・テスト 2,715 本、自作アプリ 29 本の型検査、全部緑

・自作の 3D 描画だけが、2 フレーム目で落ちた（前のフレームの値を、使い回し済みの場所から読んだ）

・規則ごと取り下げた。「822 MB → 10 MB」の改善も撤回

そのとき足した 8 行のテスト

初めて走らせた日に、別の出力先（Wasm）の、生きているバグを見つけた

700 のはずが -424242

検査は、作った瞬間から働く

2. 黙っている検査を、信じすぎない

事件簿: 印字は、ずっと正しかった

・コンパイラの印字は、出力先の番号を入れ忘れていた

・CPU エミュレータは、その番号を見ずに全部画面に出していた

・出力は正しい。テストは緑

・エミュレータを本物に近づけた瞬間、印字が壊れた（2026 年 10 月）

コードには「エミュレータは番号を無視する。QEMU は無視しない」とコメントまであった

事件簿: 本家 ruby との一致が 2,589 件

・比較には、Ruby のテスト枠組みを真似た自作の部品を使っていた

・その部品は、テストの一部を何もせずに通していた

・両側で同じ部品が動くので、両側とも同じように何もしない。数えると「一致」

・本物どおりに直したら、一致は 2,589 → 2,221。本物の穴も 10 個近く出てきた

事件簿: 道具が無ければ、成功

・QEMU との比較は「QEMU が入っていなければ飛ばす」と書いてあった

・CI の QEMU のインストールは「失敗しても赤くしない」

・どちらも単独なら妥当。重なると、検査が消えても CI は緑

・数えたら 65 本が同じ形。46 本が CI に居た

事件簿: ランダムでは、壊れない

・自作の圧縮器が、32×32 のランダムな画素の PNG で落ちた

・ランダムなバイト列 1〜4096 バイト、種 3 つでは、全部通った

・壊れたのは、PNG が実際に作るデータの形のときだけ

・その 3,104 バイトを、そのままテストに入れた

2 つの誤りが重なると、正しい出力になる

・書き戻し先がずれている × まだ誰も上書きしていない

・番号を入れ忘れた × 番号を見ない

・部品が何もしない × 両側で同じ部品

比較が見ているのは、出力だけ

3. 観察が食い違ったら、成果物を読む

事件簿: 圧縮しない圧縮器

・自作の圧縮器のテストは「元に戻るか」「検査値が合うか」だけだった

・何も圧縮しなくても、全部通る

・コンテナの層を圧縮させたら 6 分かかった。gzip は 0.1 秒

・毎回すべての距離を総当たりしていた。直して 369 秒 → 1.3 秒

事件簿: このスライドが、1 枚 25 秒

・10 月、この発表のスライドを作ったら、33 枚で 14 分かかった

・動いているプログラムを覗くと、時間は全部、あの圧縮器の総当たりの中

・直したはずの圧縮器は、PNG の部品が古い版に固定したままだった。2 つ下のライブラリ

・固定を上げて、33 枚で 3 秒。画素は 1 つも変わらない

そして、この発表の準備中にも

・9 月、証拠を測り直して「Ruby 層の 162 本一致は誤り」と訂正した

・今月、CI のログを読み直したら、誤りだったのは訂正の方だった

・162 本という数が、別の比較に 2 つあった

記憶ではなく、ログが判定した

当てるためではなく、外すために効く

1. 疑いは、検査にする —— 外れも確定する
2. 黙っている検査を、信じすぎない —— 見えない場所がある
3. 観察が食い違ったら、成果物を読む —— 成果物は構造を述べている

自分が書いていない層でも

・「下の層は正しい」は、いつでも仮定

・フレームワークでも、データベースでも、クラウドでも

・仮定が外れたときにやることは、変わらない

ありがとうございました

- Mere: https://merelang.org/
- CPU エミュレータ: https://github.com/284km/memu
- Ruby 処理系: https://github.com/284km/mere-ruby
- この資料: https://284km.github.io/decks/